JSON REFINER ADVANCED EDITION

A Professional JSON Processing and Transformation Tool using Python and Gradio

---

1. ABSTRACT

In modern software systems, data exchange plays a critical role, and JSON (JavaScript Object Notation) has emerged as a widely accepted standard for structured data representation. However, raw data collected from different sources is often inconsistent, unstructured, and difficult to process. The JSON Refiner Advanced Edition project addresses these challenges by providing a comprehensive and intelligent JSON processing solution.

This project is developed using Python and Gradio, offering a user-friendly interface for converting unstructured key-value text into structured JSON format. It incorporates advanced features such as automatic type inference, case normalization, schema validation, flattening and unflattening of JSON structures, and merging of multiple JSON objects.

The system enhances productivity by reducing manual effort, ensuring data consistency, and providing a reliable tool for developers, data analysts, and beginners. The project demonstrates the effective integration of backend processing and frontend interaction, making JSON manipulation efficient and accessible.

---

2. INTRODUCTION

2.1 Background

In the field of software development and data processing, JSON is widely used for communication between systems, especially in APIs, web applications, and cloud-based services. Despite its popularity, managing JSON data can be complex when dealing with inconsistent formats, improper data types, and nested structures.

Many users, especially beginners, face difficulties in converting raw data into valid JSON format. Manual conversion is time-consuming and error-prone. Therefore, there is a need for an automated system that simplifies JSON creation and transformation.

2.2 Problem Statement

The major problems addressed by this project include:

- Difficulty in converting raw text data into JSON
- Inconsistent key naming conventions

- Incorrect data types (string instead of integer, etc.)
- Handling nested JSON structures
- Lack of user-friendly tools for JSON processing

2.3 Proposed Solution

The JSON Refiner Advanced Edition provides a complete solution by:

- Automatically converting text into JSON
- Detecting correct data types
- Standardizing key formats
- Providing transformation operations
- Validating JSON using schema rules

---

3. OBJECTIVES

The main objectives of the project are:

- To design an automated system for JSON conversion
- To implement intelligent type inference
- To provide multiple case conversion techniques
- To validate JSON data using schema
- To implement JSON transformation techniques
- To develop an interactive user interface
- To improve data quality and consistency

---

4. SCOPE OF THE PROJECT

The scope of this project extends across multiple domains:

4.1 Academic Use

Students can use this tool to understand JSON concepts easily.

4.2 Software Development

Developers can normalize API responses and configuration files.

4.3 Data Analysis

Analysts can convert raw datasets into structured JSON for processing.

4.4 Industry Applications

- Data migration
- API integration
- Configuration management

---

5. LITERATURE REVIEW

JSON processing tools are widely available, but many existing tools lack user-friendly interfaces or advanced transformation capabilities. Traditional tools focus only on validation or formatting, while this project integrates multiple functionalities into a single platform.

This project improves upon existing solutions by combining:

- Type inference
- Case normalization
- Data transformation
- Validation
- Interactive UI

---

6. SYSTEM REQUIREMENTS

6.1 Software Requirements

- Python 3.8 or above
- Google Colab / Jupyter Notebook
- Gradio framework
- jsonschema library
- json5 library

6.2 Hardware Requirements

- Processor: Intel i3 or higher
- RAM: Minimum 2GB
- Storage: 500MB free space

- Internet connection

---

7. SYSTEM ARCHITECTURE

The system follows a modular architecture consisting of the following components:

7.1 Input Module

Accepts user input in key-value format.

7.2 Processing Engine

- Parses input text
- Converts into JSON
- Performs type inference

7.3 Transformation Module

- Case conversion
- Flattening and unflattening
- Merging JSON objects

7.4 Validation Module

- Checks required fields
- Validates JSON schema

7.5 Output Module

Displays formatted JSON and validation results.

---

8. METHODOLOGY

The project follows a step-by-step processing methodology:

Step 1: Data Input

User provides key-value pairs.

Step 2: Parsing

The system separates keys and values.

Step 3: Type Inference

Values are converted into appropriate types:

- Integer
- Float
- Boolean
- Null
- String

Step 4: Key Normalization

Keys are converted into selected case styles.

Step 5: Data Transformation

- Removal of null values
- Flattening nested structures
- Merging multiple JSON objects

Step 6: Validation

- Required fields checking
- Schema validation

Step 7: Output Generation

Formatted JSON is displayed to the user.

---

9. MODULE DESCRIPTION

9.1 Type Inference Module

Automatically detects the correct data type for each value.

9.2 Case Conversion Module

Supports:

- snake_case
- camelCase
- PascalCase
- kebab-case

9.3 Validation Module

Ensures data correctness using schema rules.

9.4 Transformation Module

Includes:

- Flatten
- Unflatten
- Merge
- Remove null values

9.5 User Interface Module

Developed using Gradio for interactive usage.

---

10. FEATURES OF THE SYSTEM

- Automatic JSON generation
- Intelligent type detection
- Multiple case conversion styles
- Schema validation
- Data transformation tools
- Interactive UI
- Error handling and reporting

---

11. SAMPLE INPUT AND OUTPUT

Input

name: John

age: 28

active: true

balance: 1500.50

Output

```
{
  "name": "John",
  "age": 28,
```

"active": true,

"balance": 1500.5

}

---

12. ADVANTAGES

- Reduces manual effort
- Improves accuracy
- Easy to use
- Supports multiple operations
- Suitable for beginners and professionals

---

13. LIMITATIONS

- Limited handling of complex nested input
- Requires proper input format
- Internet required for deployment

---

14. FUTURE ENHANCEMENTS

- Machine learning-based inference
- Database integration
- Batch processing
- API development
- Cloud integration

---

15. REFERENCES

1. Python Official Documentation
2. Gradio Documentation
3. JSON.org

---

16. PROGRAM

```
#  JSON Refiner Program (Main Functions Only)
import gradio as gr
import json
import json5
from jsonschema import validate, ValidationError

# ====================================
# CORE JSON REFINER CLASS
# ====================================

class JSONRefiner:

  # -------- TYPE INFERENCE --------
  def infer_type(self, value):
    value = value.strip()

    if value.lower() in ["true", "yes"]:
      return True
    if value.lower() in ["false", "no"]:
      return False
    if value.lower() in ["null", "none"]:
      return None
```

```
    try:
        if "." in value:
            return float(value)
        return int(value)
    except:
        pass

    # Try JSON object/array
    try:
        return json.loads(value)
    except:
        return value

# -------- PARSE TEXT --------
def parse_text(self, text, case_style):
    data = {}
    errors = []

    lines = text.split("\n")

    for i, line in enumerate(lines, 1):
        if ":" not in line:
            continue

        key, value = line.split(":", 1)
        key = key.strip()
        value = value.strip()
```

```
        key = self.convert_key(key, case_style)

        if key in data:
            errors.append(f"Duplicate key at line {i}: {key}")
            continue

        data[key] = self.infer_type(value)

    return data, errors

# -------- CASE CONVERSION --------
def convert_key(self, key, style):
    if style == "snake_case":
        return key.lower().replace(" ", "_")

    if style == "camelCase":
        words = key.split()
        return words[0].lower() + "".join(w.capitalize() for w in words[1:])

    if style == "PascalCase":
        return "".join(word.capitalize() for word in key.split())

    if style == "kebab-case":
        return key.lower().replace(" ", "-")

    return key

# -------- REMOVE NULL --------
```

```
def remove_nulls(self, data):
    return {k: v for k, v in data.items() if v is not None}

# -------- FLATTEN --------
def flatten(self, data, parent=""):
    items = {}

    for k, v in data.items():
        new_key = f"{parent}.{k}" if parent else k

        if isinstance(v, dict):
            items.update(self.flatten(v, new_key))
        else:
            items[new_key] = v

    return items

# -------- UNFLATTEN --------
def unflatten(self, data):
    result = {}

    for key, value in data.items():
        parts = key.split(".")
        temp = result

        for part in parts[:-1]:
            if part not in temp:
                temp[part] = {}
```

```
            temp = temp[part]

        temp[parts[-1]] = value

    return result

# -------- MERGE --------
def merge(self, j1, j2):
    result = j1.copy()
    result.update(j2)
    return result

# -------- VALIDATION --------
def validate_json(self, data, schema_text):
    if not schema_text.strip():
        return "No schema provided"

    try:
        schema = json.loads(schema_text)
        validate(instance=data, schema=schema)
        return "✅ Schema valid"
    except ValidationError as e:
        return f"❌ {e.message}"
    except:
        return "❌ Invalid schema"

refiner = JSONRefiner()
```

```
# ====================================
# FUNCTIONS FOR UI
# ====================================

def refine(text, case_style, remove_null, flatten):
    data, errors = refiner.parse_text(text, case_style)

    if remove_null:
        data = refiner.remove_nulls(data)

    if flatten:
        data = refiner.flatten(data)

    output = json.dumps(data, indent=2)

    msg = "✅ Success"
    if errors:
        msg += "\n" + "\n".join(errors)

    return output, msg

def validate_func(json_text, schema_text):
    try:
        data = json.loads(json_text)
        result = refiner.validate_json(data, schema_text)
        return result
    except:
        return "❌ Invalid JSON"
```

```
def case_convert(json_text, style):
    try:
        data = json.loads(json_text)
        new_data = {}

        for k, v in data.items():
            new_key = refiner.convert_key(k, style)
            new_data[new_key] = v

        return json.dumps(new_data, indent=2)
    except:
        return "Error"

def merge_func(j1, j2):
    try:
        return json.dumps(refiner.merge(json.loads(j1), json.loads(j2)), indent=2)
    except:
        return "Error"

def flatten_func(json_text):
    return json.dumps(refiner.flatten(json.loads(json_text)), indent=2)

def unflatten_func(json_text):
    return json.dumps(refiner.unflatten(json.loads(json_text)), indent=2)

# ====================================
# GRADIO UI (MULTI-TAB)
```

```
# ====================================

with gr.Blocks() as app:
  gr.Markdown("# 🚀 JSON Refiner Professional Tool")

  with gr.Tabs():

    # TAB 1
    with gr.TabItem("Refine"):
      text = gr.Textbox(lines=10, label="Input Text")
      case = gr.Radio(["snake_case", "camelCase", "PascalCase", "kebab-case"],
value="snake_case")
      remove_null = gr.Checkbox(label="Remove Null")
      flat = gr.Checkbox(label="Flatten")

      btn = gr.Button("Refine")

      out = gr.Code()
      msg = gr.Textbox()

      btn.click(refine, [text, case, remove_null, flat], [out, msg])

    # TAB 2
    with gr.TabItem("Validation"):
      json_in = gr.Code()
      schema = gr.Code()
      btn2 = gr.Button("Validate")
      result = gr.Textbox()
```

```
    btn2.click(validate_func, [json_in, schema], result)

# TAB 3
with gr.TabItem("Case Convert"):
    json_case = gr.Code()
    style = gr.Radio(["snake_case", "camelCase", "PascalCase", "kebab-case"])
    btn3 = gr.Button("Convert")
    out_case = gr.Code()

    btn3.click(case_convert, [json_case, style], out_case)

# TAB 4
with gr.TabItem("Merge"):
    j1 = gr.Code()
    j2 = gr.Code()
    btn4 = gr.Button("Merge")
    out_merge = gr.Code()

    btn4.click(merge_func, [j1, j2], out_merge)

# TAB 5
with gr.TabItem("Flatten"):
    j_flat = gr.Code()
    btn5 = gr.Button("Flatten")
    out_flat = gr.Code()

    btn5.click(flatten_func, j_flat, out_flat)
```

```
    # TAB 6
    with gr.TabItem("Unflatten"):
      j_unflat = gr.Code()
      btn6 = gr.Button("Unflatten")
      out_unflat = gr.Code()

      btn6.click(unflatten_func, j_unflat, out_unflat)

app.launch()
```

---

17. OUTPUT

. Output – Refine Operation

Input:

name: John

age: 28

active: true

balance: 1500.50

city: Chennai

output:


```
{
  "name": "John",
  "age": 28,
  "active": true,
  "balance": 1500.5,
  "city": "Chennai"
}
```

Status:

✅ **Success**

🔹 **Output – Remove Null Option**

Input:

name: John

age: 28

email: null

Output:

```
{
  "name": "John",
  "age": 28
}
```

🔹 **Output – Flatten Operation**

Input:

```
{
  "user": {
    "name": "John",
    "age": 28
  }
}
```

Output:

```
{
  "user.name": "John",
  "user.age": 28
}
```

🔹 **Output – Unflatten Operation**

Input:

```
{
  "user.name": "John",
  "user.age": 28
}
```

Output:

```
{
  "user": {
    "name": "John",
    "age": 28
  }
}
```

🔹 **Output – Merge Operation**

Input:

JSON 1:

```
{
  "name": "John"
}
```

JSON 2:

```
{
  "age": 28
}
```

Output:

```
{
  "name": "John",
```

```
  "age": 28
}
```

🔹 **Output – Case Conversion**

Input:

```
{
  "First Name": "John",
  "Last Name": "Doe"
}
```

Output (snake_case):

```
{
  "first_name": "John",
  "last_name": "Doe"
}
```

🔹 **Output – Validation**

Valid JSON:

✅ **Schema valid**

Invalid JSON:

❌ **'age' is a required property**

---

18. CONCLUSION

The JSON Refiner Advanced Edition project successfully demonstrates how data processing can be simplified using automation and intelligent techniques. The integration of Python and Gradio enables the development of a powerful and user-friendly system.

The project not only enhances productivity but also ensures data consistency and accuracy. It serves as a valuable tool for both academic and professional applications.